# A Sample Article Using IEEEtran.cls for IEEE Journals and Transactions

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—This document describes the most common article elements and how to use the IEEEtran class with LaTeX to produce files that are suitable for submission to the IEEE. IEEEtran can produce conference, journal, and technical note (correspondence) papers with a suitable choice of class options.**

***Index Terms*—Article submission, IEEE, IEEEtran, journal, LaTeX, paper, template, typesetting.**

## I. INTRODUCTION

Multi-robot systems (MRS) have been widely used in applications such as persistent surveillance [1], logistics [2], and autonomous maintenance [3], where multiple robots can operate concurrently and cooperatively to improve efficiency and accomplish tasks that are beyond the capability of a single robot. In practice, many multi-robot tasks are subject to temporal requirements. For example, in warehouse logistics, mobile robots may first retrieve a shelf and bring it to a picking station, where item picking can begin only after the shelf has arrived. Such task dependencies naturally introduce temporal constraints such as ordering, persistence, and synchronization. To specify these constraints in a rigorous way, many recent work have used formal languages such as linear temporal logic (LTL) [4]; see, for example, [5], [6].

In this work, we consider the multi-robot task allocation (MRTA) problem under temporal logic specifications, referred to as LTL-MRTA, which was formalized in [7]. In LTL-MRTA, the task is specified by an LTL formula, but the specific robots assigned to each subtask are not predetermined by the formula. For example, the specification may require that two type-1 robots first complete a pickup-and-delivery task and then wait at the delivery site until one type-2 robot arrives, without specifying which exact robots should be chosen. This problem becomes increasingly difficult as the number of robots, the workspace size, and the complexity of the task specification grow, since the associated search and optimization space may expand explosively. As a result, existing approaches, such as mixed-integer linear programming (MILP)-based methods [7], [8] and search-based methods [9], [10] can be computationally demanding in large-scale problems, even when only an initial feasible solution is required. In practical robotic applications, however, it is often important to obtain an executable solution early, especially in large-scale settings where task execution needs to start promptly and decisions may need to be updated online. Planning-tree-based [6], [11] and partial-order-based methods [3] highlighted the practical importance of fast first-solution generation. However, planning-tree-based methods may be prone to local optima, while partial-order-based methods may be less effective for complex tasks due to the need to extract the partial-order relations among subtasks. Consequently, for LTL-MRTA, developing methods that can quickly return a feasible solution and further improve it toward optimality remains an important research direction.

To address this problem, we propose a novel framework that integrates robot planning into the automaton generation process. Unlike existing methods that first construct the complete NBA and then perform planning on top of it, our approach introduces information about the robot team, the workspace, and intermediate planning results during the translation from LTL to automata. Specifically, we follow the translation pipeline of LTL2BA [12], a classical tool widely used in the literature [13], [14], which translates an LTL formula into a very weak alternating automaton (VWAA), then into a generalized Büchi automaton (GBA), and finally into a nondeterministic Büchi automaton (NBA). During the GBA construction stage, we prune branches that are irrelevant to task planning, so as to eliminate unpromising search directions early. Then, during the NBA construction stage, we simultaneously generate the NBA and a planning graph, which allows us to quickly obtain an initial feasible solution and further prune the search space using the information provided by that solution. Finally, based on the generated planning graph, we apply a warm-start branch-and-bound to improve the solution quality. In this way, the proposed method supports rapid first-solution generation as well as continued improvement toward better solutions.

### A. Related Work

Existing planning methods under LTL constraints can generally be classified into bottom-up and top-down approaches. Bottom-up approaches usually formulate the problem as a task-coordination problem under local LTL specifications, where each robot is assigned a local LTL task and team behavior is achieved through communication, synchronization, and online replanning. For example, Guo et al. [15] studied a decentralized planning framework under local LTL specifications with online plan reconfiguration, while Yu et al. [16] developed a distributed motion-coordination strategy under local LTL specifications, where conflicts among neighboring robots are detected and resolved online to ensure safety and task satisfaction. In contrast, top-down approaches start from a global LTL formula that specifies the overall team mission. This paper focuses on the latter setting.

For top-down methods, a common strategy is to construct the product automaton of NBA translated from the global

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

LTL formula and the discrete transition systems of the robots and then solve the resulting graph-search problem [9], [17]. However, this approach suffers from state explosion , because the number of states in the product automaton increases rapidly with the workspace/state-space size, the number of robots, and the complexity of the task, as reflected by the size of the automaton derived from the specification. To alleviate this issue, sampling-based methods [10], [18] incrementally construct trees that approximate the state space and transitions of the synchronous product automaton, instead of explicitly building the full product automaton. This significantly improves scalability while retaining probabilistic completeness and asymptotic optimality. However, such methods typically require the global LTL formula to explicitly assign subtasks to robots. When the robot assignment is not given a priori, a possible workaround is to enumerate all admissible robot combinations and connect them using OR operators, but this can result in exponentially long LTL formulas.

To avoid the combinatorial burden and reduced flexibility caused by explicitly allocate tasks to robots, some works decompose the global LTL specifications. For example, [14], [19] proposed the simultaneous task allocation and planning (STAP) framework, which constructs a team model that integrates task decomposition and task allocation. By decomposing the global task into parallelizable subtasks, the framework improves planning efficiency. However, these works do not explicitly address tightly coupled collaborative tasks requiring the simultaneous participation of multiple robots. Other works address global temporal-logic task planning by using mixed-integer linear programming (MILP) [7], [8], [20]. In particular, to mitigate the impact of workspace size on the computational burden, [7] proposed a hierarchical architecture that separates task planning from motion planning. Unfortunately, MILP-based methods can become computationally demanding in large-scale settings, even when only a first feasible solution is required. More recently, Luo et al [21] introduced a hierarchical temporal-logic representation, H-LTL$_f$ and improves the interpretability and planning efficiency for complex tasks.

Planning-decision-tree-based methods [11], [22] have been proposed to improve the speed of temporal-logic mission planning. These methods represents task progression and task allocation incrementally, enabling real-time planning for large-scale multi-robot systems. Although such methods are highly efficient in generating feasible solutions, they typically rely on greedy task-allocation decisions, and therefore can be prone to getting trapped in local optima. Liu et al. [3] analyze the task automaton to extract partial-order relations among subtasks, and then use a branch-and-bound search procedure to compute task assignments. Their method explicitly emphasizes the importance of obtaining a valid plan early: it is formulated as an anytime algorithm, so that a feasible solution can be returned quickly and then progressively improved as computation continues. However, when the task specification becomes more complex, the efficiency of obtaining the first feasible solution may decrease, since extracting the partial-order relations among subtasks can itself take additional time. As illustrated in the paper, for an automaton with hundreds of states, the algorithm may require tens or even hundreds of seconds to obtain the first partial order.

Overall, while existing top-down methods have advanced LTL-MRTA from different perspectives, they still struggle to simultaneously achieve rapid first-solution generation and effective subsequent optimization in large-scale problems. This limitation is particularly evident when substantial computation is required before a planning structure suitable for allocation can even be obtained. Different from methods that plan after constructing the full automaton, our approach incorporates robot, workspace, and intermediate planning information directly into the automaton-generation process. By pruning planning-irrelevant branches early, incrementally constructing an NBA and planning graph, and then warm-starting branch-and-bound from the resulting feasible solution, the proposed framework supports both fast initial planning and continued improvement toward higher-quality solutions.

### B. Contributions

The main contributions of this work are summarized as follows. First, we develop a new framework for LTL-MRTA that tightly couples task planning with the automaton-generation process, instead of treating automaton construction and planning as two separate stages. By introducing robot, workspace, and intermediate planning information into the translation procedure, the proposed method is able to better align the generated automaton structure with the downstream planning problem. Second, we propose a planning-oriented pruning strategy during automaton construction. In particular, branches that are irrelevant or unpromising for task planning are removed early in the GBA stage, and an NBA is then constructed together with a planning graph in an incremental manner. This design allows the method to produce an initial feasible solution quickly while further reducing the search space using information revealed by that solution. Third, we develop a warm-start branch-and-bound procedure on the generated planning graph to further improve solution quality after a feasible plan is found. Therefore, the proposed framework explicitly addresses two practically important objectives in LTL-MRTA: rapid first-solution generation and continued improvement of the final plan quality. Experimental results demonstrate the effectiveness of the proposed method, especially in complex tasks and large-scale problem instances.

## II. PRELIMINARIES

### A. Linear Temporal Logic

Linear temporal logic (LTL) is widely used to specify temporal tasks over a set of atomic propositions $AP$. Following the standard syntax, an LTL formula over $AP$ is recursively defined by

$$\varphi := \top \mid \pi \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \bigcirc\varphi \mid \varphi_1 \,\mathcal{U}\, \varphi_2,$$

where $\pi \in AP$, $\top$ denotes the constant true, $\bigcirc$ is the next operator, and $\mathcal{U}$ is the until operator. Other temporal operators can be derived in the standard way, including eventually $\Diamond\varphi := \top\,\mathcal{U}\,\varphi$, always $\Box\varphi := \neg\Diamond\neg\varphi$.

Let $\Sigma = 2^{AP}$ be the alphabet. An infinite word over $\Sigma$ is a sequence $w = \sigma_0\sigma_1\cdots \in \Sigma^\omega$, where each $\sigma_k \in \Sigma$ denotes

the set of atomic propositions satisfied at step $k$. The notation $w \models \varphi$ means that the word $w$ satisfies the LTL formula $\varphi$. The language of $\varphi$ is defined as $\mathrm{Words}(\varphi) := \{ w \in \Sigma^\omega \mid w \models \varphi \}$.

It is well known that any satisfiable LTL formula can be translated into a nondeterministic Büchi automaton (NBA).

**Definition 1** (Nondeterministic Büchi Automaton)**.** A nondeterministic Büchi automaton (NBA) is a tuple

$$\mathcal{B} = (Q_B, Q_{B,0}, \Sigma_B, \rightarrow_B, Q_{B,F}), \tag{1}$$

where $Q_B$ is a finite set of states, $Q_{B,0} \subseteq Q_B$ is the set of initial states, $\Sigma_B$ is the alphabet, $\rightarrow_B \subseteq Q_B \times \Sigma_B \times Q_B$ is the transition relation, and $Q_{B,F} \subseteq Q_B$ is the set of accepting states.

An infinite run of $\mathcal{B}$ over a word $w = \sigma_0\sigma_1 \cdots$ is a sequence $\rho = q_0 q_1 \cdots$ such that $q_0 \in Q_{B,0}$ and $(q_k, \sigma_k, q_{k+1}) \in \rightarrow_B$ for all $k \geq 0$, where each $\sigma_k \in \Sigma_B$. The run is accepting if it visits accepting states infinitely often, i.e., $\mathrm{Inf}(\rho) \cap Q_{B,F} \neq \emptyset$. For a satisfiable LTL formula, an accepting run can be written in the prefix–suffix form, where the prefix reaches an accepting state once and the suffix repeats a cycle around that accepting state indefinitely.

### B. LTL2BA Translation

In this work, we follow the translation pipeline of LTL2BA [12] to construct automata from LTL specifications. Given an LTL formula $\varphi$, LTL2BA first rewrites $\varphi$ into negative normal form (NNF), so that negation only appears in front of atomic propositions. It then applies rewriting rules to simplify the formula and reduce the number of temporal operators before automaton generation. An abstract syntax tree is then generated from the simplified formula.

**Definition 2** (Abstract Syntax Tree)**.** Given an LTL formula $\varphi$ in negative normal form, its abstract syntax tree (AST), denoted by $\mathcal{T}_\varphi$, is a rooted ordered tree in which each internal node is labeled by a logical or temporal operator, and each leaf is labeled by an atomic proposition or its negation. The recursive structure of $\mathcal{T}_\varphi$ represents the syntactic decomposition of $\varphi$ and provides the basis for extracting temporal subformulas during automaton construction.

Starting from the AST, LTL2BA first constructs a very weak alternating automaton (VWAA). An alternating automaton may require multiple successor states to hold simultaneously under the same input symbol. Moreover, the "very weak" property imposes a partial order on states, so that transitions can only remain at the same level or move downward in this order, which yields a simpler automaton structure.

**Definition 3** (Very Weak Alternating Automaton)**.** A very weak alternating automaton (VWAA) is a tuple

$$\mathcal{V}_\varphi = (Q_v, \Sigma, \delta_v, I_v, F_v, \preceq_v),$$

where $Q_v$ is a finite set of states, $\Sigma$ is the input alphabet, $\delta_v$ is the transition function, $I_v$ is the initial condition, $F_v \subseteq Q_v$ is the set of accepting states, and $\preceq_v$ is a partial order on $Q_v$ such that every state appearing in the transition of a state $q_v \in Q_v$ is lower than or equal to $q_v$ under $\preceq_v$.

Based on the VWAA, LTL2BA then constructs a generalized Büchi automaton (GBA), which preserves multiple acceptance requirements in a compact intermediate form before the final degeneralization step.

**Definition 4** (Generalized Büchi Automaton)**.** A generalized Büchi automaton (GBA) is a tuple

$$\mathcal{G}_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, \mathcal{F}_g),$$

where $Q_g$ is a finite set of states, $Q_{g,0} \subseteq Q_g$ is the set of initial states, $\Sigma$ is the input alphabet, $\rightarrow_g \subseteq Q_g \times \Sigma \times Q_g$ is the transition relation, and $\mathcal{F}_g = \{F_{g,1}, \ldots, F_{g,m}\}$ is a finite collection of accepting transition sets with $F_{g,i} \subseteq \rightarrow_g$ for all $i \in \{1, \ldots, m\}$. A run is accepting if, for every $F_{g,i} \in \mathcal{F}_g$, it traverses transitions in $F_{g,i}$ infinitely often.

After constructing the GBA, LTL2BA applies a standard degeneralization step to obtain the final nondeterministic Büchi automaton (NBA), denoted by $\mathcal{B}_\varphi$. Therefore, the overall translation pipeline can be summarized as

$$\varphi \;\rightarrow\; \mathcal{T}_\varphi \;\rightarrow\; \mathcal{V}_\varphi \;\rightarrow\; \mathcal{G}_\varphi \;\rightarrow\; \mathcal{B}_\varphi.$$

A key advantage of LTL2BA is that simplifications are performed throughout the translation process. Besides simplifying the formula before automaton generation, the algorithm also simplifies intermediate automata on-the-fly by removing inaccessible states, eliminating implied transitions, and merging equivalent states whenever possible. These operations reduce the number of intermediate states and transitions and improve both time and memory efficiency. In this paper, we build on this translation pipeline and further incorporate planning-related information during automaton construction so that the generated automaton structure is better aligned with the downstream task-planning problem.

## III. PROBLEM DESCRIPTION

### A. Workspace and Heterogeneous Robot Team

Consider a discrete workspace $\mathcal{W}$ containing a finite set of regions of interest $\mathcal{L} = \{l_1, l_2, \ldots, l_N\}$. let $\mathcal{O} = \{o_1, o_2, \ldots, o_M\}$ denote the non-interested regions such as obstacles and forbidden regions. We assume that regions of interest do not overlap with each other and regions of non-interest , i.e., $l_i \cap o_j = \emptyset, \quad \forall i \in \{1, \ldots, N\}, \ \forall j \in \{1, \ldots, M\}$, $l_i \cap l_j = \emptyset, \quad \forall i \neq j$.

A team of heterogeneous robots is denoted by $\mathcal{R} = \{r_1, r_2, \ldots, r_M\}$. The robots belong to a finite set of types $\mathcal{T} = \{1, 2, \ldots, m\}$, where each type represents a specific capability, such as sensing, manipulation, or mobility. Each robot belongs to exactly one type. Let $\tau(r_k) \in \mathcal{T}$ denote the type of robot $r_k$, and let $\mathcal{R}_j = \{r_k \in \mathcal{R} \mid \tau(r_k) = j\}$ be the set of robots of type $j$. The position of robot $r_k$ is denoted by $x(r_k)$ and the velocity of $r_k$ is denoted by $v(r_k)$.

### B. Task Specification

Before formulating the global temporal logic task, we introduce two classes of atomic propositions. (i) For each region $l_i \in \mathcal{L}$, let the proposition $p_i$ is true, if at least one robot is located in region $l_i$. Let $\mathcal{P} \triangleq \{p_i \mid l_i \in \mathcal{L}\}$. (ii) Let $a_i$ be an action proposition. For example, $a_i$ can represent a group of robots visits a specified target region. Let $\mathcal{A} \triangleq \{a_1, a_2, \ldots, a_{N_q}\}$ denote the set of all action propositions. Each $a_i \in \mathcal{A}$ is associated with an execution requirement $c(a_i) = (\tau_i, n_i, l_i)$, where $\tau_i \in \mathcal{T}$ is the required robot type, $n_i \in \mathbb{N}$ is the required number of robots of type $\tau_i$, and $l_i \in \mathcal{L}$ is the target region. Thus, the proposition $a_i$ is true, i.e., $a_i = \top$, if and only if the corresponding action is performed by $n_i$ robots of type $\tau_i$ at region $l_i$. The complete set of atomic propositions used in the temporal logic specification is defined as $\mathcal{AP} \triangleq \mathcal{P} \cup \mathcal{A}$.

**Example 1.** For example, consider two robot types $\mathcal{T} = \{1, 2\}$, two regions of interest $\mathcal{L} = \{l_1, l_2\}$. The corresponding region propositions include $p_1, \quad p_2$. Let $a_1$ and $a_2$ be two action propositions with $c(\pi_1) = (2, 2, l_1), \quad c(\pi_2) = (1, 1, l_2)$.

### C. Problem Definition

Consider an LTL formula $\varphi$ over $\mathcal{AP}$, and let $\rho_\varphi$ be a prefix–suffix accepting run of its associated NBA, written as$\rho_\varphi = \rho_\varphi^{\mathrm{pre}}\big(\rho_\varphi^{\mathrm{suf}}\big)^\omega$. For the prefix–suffix run, we use the finite sequence $\Pi_\varphi^{\mathrm{fin}} = \Pi_\varphi^{\mathrm{pre}}\Pi_\varphi^{\mathrm{tra}}\Pi_\varphi^{\mathrm{suf}}$ to represent one completed execution. The segment $\Pi_\varphi^{\mathrm{pre}}$ corresponds to the prefix part that reaches an accepting state. The segment $\Pi_\varphi^{\mathrm{tra}}$, if nonempty, is the path from this accepting state to the beginning of the selected suffix cycle. The segment $\Pi_\varphi^{\mathrm{suf}}$ corresponds to one traversal of that suffix cycle. When the selected suffix cycle starts from the accepting state reached by the prefix,$\Pi_\varphi^{\mathrm{tra}}$ is empty. Let $T(\Pi_\varphi^{\mathrm{fin}})$ denote the completion time of $\Pi_\varphi^{\mathrm{fin}}$. The cost is defined as

$$J_\varphi = T(\Pi_\varphi^{\mathrm{fin}}). \tag{2}$$

**Problem III.1.** Consider a discrete workspace $W$ with regions of interest, obstacles, and forbidden regions, a heterogeneous robot team $R$ with type set $T$, and a valid LTL formula $\varphi$ defined over the atomic proposition set $\mathcal{AP} = P \cup A$. Plan a motion and action sequence for each robot $r_k \in R$, and assign robots to the required action propositions such that the finite trace generated by the team execution satisfies $\varphi$ and all action requirements $c(a_i) = (\tau_i, n_i, l_i)$ are fulfilled, and the completion time $T_\varphi$ is minimized.

## IV. System Overview

## V. Pruning for GBA

In this section, we introduce the proposed pruning procedure during the construction of GBA. Unlike methods that perform pruning after the complete NBA has been constructed, the proposed method prunes the GBA both during and after the VWAA-to-GBA construction stage. This early pruning reduces the size of the GBA and prevents unnecessary transitions from being propagated to the subsequent NBA construction and planning stages. The proposed GBA pruning consists of the following two parts.

*1) Infeasible Transition Pruning During GBA Construction:* In the VWAA-to-GBA construction, a GBA state $q_g \in Q_g$ corresponds to a conjunction of VWAA states, i.e. represents a set of VWAA states that must hold simultaneously. Suppose that $q_g = q_{v,1} \wedge q_{v,2} \wedge \cdots \wedge q_{v,n}$, where $q_{v,i} \in Q_v$. For each VWAA state $q_{v,i}$, its transition $\delta_v(q_{v,i})$ provides possible ways to move from $q_{v,i}$ to a set of successor VWAA states under input conditions. To construct an outgoing GBA transition from $q_g$, one transition is selected from each $\delta_v(q_{v,i})$, and these selected VWAA transitions are combined. Their input conditions are conjoined, and their successor VWAA states are collected to form the successor GBA state $q_g' \in Q_g$. Thus, each such combination produces a GBA transition $q_g \xrightarrow{\sigma} q_g'$, where $\sigma \in \Sigma$ denotes the input symbol satisfying the combined transition condition. The accepting structure is also transferred during this construction. The co-Büchi accepting states $F_v$ of the VWAA are transformed into the collection of accepting transition sets $F_g = \{F_{g,1}, \ldots, F_{g,m}\}$ of the GBA. Therefore, GBA acceptance is checked on transitions rather than on states: a run of $G_\varphi$ is accepting if it traverses transitions in every accepting transition set in $F_g$ infinitely often.

Building on the above construction, we further introduce infeasible transition pruning during GBA generation to eliminate transitions that cannot be realized by the considered multi-robot system. Specifically, for a GBA transition, if no robot combination in $\mathcal{R}$ can satisfy its transition condition, then this transition is called an infeasible transition. For example, suppose that a transition requires three type-1 robots to execute an action at region $l_1$ and, at the same time, another three type-1 robots to execute an action at region $l_2$. If the robot team $\mathcal{R}$ contains only five type-1 robots in total, then no robot combination can satisfy this transition condition. Therefore, this transition is infeasible and can be removed during GBA construction. During the construction, infeasible transition pruning is performed when outgoing GBA transitions are being generated. For a GBA state $q_g$, the outgoing transitions are obtained by combining the outgoing transitions of the VWAA states. Before this transition is inserted into $\rightarrow_g$, we check whether its transition condition can be satisfied by the robot team $\mathcal{R}$. If no robot combination in $\mathcal{R}$ can satisfy the condition, the candidate transition is identified as infeasible and is discarded immediately. Otherwise, the transition is kept, and the GBA construction continues.

*2) Pruning of Decomposition-Inducing Transitions:* The second part of the proposed GBA pruning is performed after the GBA has been constructed. This step removes direct transitions that impose unnecessary synchronization among subtasks. Due to the concurrent nature of multi-robot systems, forcing several independent subtasks to be satisfied at the same automaton step may be unnecessarily restrictive. If the same automaton progress can be achieved by satisfying these subtasks through multiple smaller transitions, then the direct transition requiring their simultaneous satisfaction is unnecessary for planning. Therefore, this pruning aims to remove such direct composite transitions while preserving their decomposed alternatives.

Based on this observation, after the GBA is generated, we search for decomposition-inducing transitions. Specifically, a GBA transition is removed if it can be translated to a decomposable transition in the resulting NBA.

**Definition 5** (Decomposable transition)**.** Consider the NBA $B_\varphi$. A transition from state $q_i$ to state $q_j$ is called decomposable if there exists another state $q_k$ such that, for any input symbols $\sigma_{ik}, \sigma_{kj} \in \Sigma$, if

$$q_k \in \delta_B(q_i, \sigma_{ik}) \quad \text{and} \quad q_j \in \delta_B(q_k, \sigma_{kj}),$$

then

$$q_j \in \delta_B(q_i, \sigma_{ik} \cup \sigma_{kj}).$$

**Definition 6** (Decomposition-inducing transition)**.** Consider a GBA transition

$$e_g = (q_g, \sigma, q_g') \in \rightarrow_g,$$

where $q_g, q_g' \in Q_g$ and $\sigma \in \Sigma$. The transition $e_g$ is called decomposition-inducing if, after the GBA $G_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, F_g)$ is transformed into the corresponding NBA $B_\varphi$, the NBA transition induced by $e_g$ is decomposable in the sense of Definition 5.

To identify decomposition-inducing transitions directly in the GBA, two conditions need to be satisfied. First, the transition must be decomposable. Specifically, consider a transition from $q_{g,i}$ to $q_{g,j}$ in $G_\varphi$. This transition is called decomposable in the GBA if there exists an intermediate state $q_{g,k} \in Q_g$ such that, for any input symbols $\sigma_{ik}, \sigma_{kj} \in \Sigma$, if $q_{g,k} \in \delta_g(q_{g,i}, \sigma_{ik})$ and $q_{g,j} \in \delta_g(q_{g,k}, \sigma_{kj})$, then $q_{g,j} \in \delta_g(q_{g,i}, \sigma_{ik} \cup \sigma_{kj})$. Second, the final set consistency condition must hold. Consider the decomposed path $q_{g,i} \xrightarrow{\sigma_{ik}} q_{g,k} \xrightarrow{\sigma_{kj}} q_{g,j}$ and the corresponding direct transition $q_{g,i} \xrightarrow{\sigma_{ik} \cup \sigma_{kj}} q_{g,j}$. Since the acceptance condition of a GBA is defined on transitions, each GBA transition may belong to one or more accepting sets. For a transition $e_g \in \rightarrow_g$, we define its final set as

$$\mathrm{Fin}(e_g) = \{F_{g,h} \in F_g \mid e_g \in F_{g,h}\},$$

where $F_g = \{F_{g,1}, \ldots, F_{g,m}\}$ is the collection of accepting sets of the GBA. Let $e_{ik} = (q_{g,i}, \sigma_{ik}, q_{g,k})$, $e_{kj} = (q_{g,k}, \sigma_{kj}, q_{g,j})$, and $e_{ij} = (q_{g,i}, \sigma_{ik} \cup \sigma_{kj}, q_{g,j})$. The final-set consistency condition requires that

$$\mathrm{Fin}(e_{ij}) = \mathrm{Fin}(e_{ik}) \cup \mathrm{Fin}(e_{kj}).$$

This condition means that the direct transition $e_{ij}$ and the decomposed path $e_{ik}e_{kj}$ satisfy exactly the same GBA accepting transition sets. Therefore, replacing $e_{ij}$ with the two-hop path does not change the acceptance information used in the later GBA-to-NBA transformation.

*Lemma* V.1. *The decomposition-inducing transitions removed in the GBA are the decomposable transitions in the NBA.*

*Proof.* In the GBA-to-NBA construction, each NBA state is uniquely represented as a pair $(q_g, \ell)$, where $q_g$ is a GBA state and $\ell \in \{0, 1, \ldots, m_g\}$ denotes the current acceptance progress, with $m_g = |F_g|$. The value $\ell$ indicates how many accepting transition sets have been satisfied in the prescribed order. Given a transition $e_g = (q_g, \sigma, q_g') \in \rightarrow_g$, define its final set as $\mathrm{Fin}(e_g) = \{h \in \{1, \ldots, m_g\} \mid e_g \in F_{g,h}\}$. When $e_g$ is traversed from the NBA state $(q_g, \ell)$, the acceptance progress is updated by$\ell' = \mathrm{Next}(\ell, \mathrm{Fin}(e_g))$, where $\mathrm{Next}(\ell, \mathrm{Fin}(e_g)) = \max\{r \in \{\ell, \ldots, m_g\} \mid \{\ell+1, \ell+2, \ldots, r\} \subseteq \mathrm{Fin}(e_g)\}$. Thus, the successor NBA state induced by $e_g$ is $(q_g', \ell')$.

In the proposed GBA pruning, a removable direct transition must satisfy not only the decomposability condition, but also the final-set consistency condition. That is, for a direct transition $e_{ij}$ and its decomposed two-hop path $e_{ik}e_{kj}$, we require $\mathrm{Fin}(e_{ij}) = \mathrm{Fin}(e_{ik}) \cup \mathrm{Fin}(e_{kj})$. This condition ensures that replacing $e_{ij}$ by the two-hop path produces the same acceptance progress in the GBA-to-NBA construction. Consequently, the replacement does not introduce additional NBA state splitting, and the transition induced by $e_{ij}$ remains decomposable in the resulting NBA. □

## VI. On-the-Fly Plan Graph Construction

In the LTL2BA translation pipeline, after the GBA has been obtained, the final NBA is constructed from the GBA. In the original LTL2BA implementation, this procedure follows a DFS order, which expands newly generated NBA states according to their original generation order. In this work, we modify this process by introducing planning-aware lower-bound priorities into the NBA construction. More specifically, when an NBA state is expanded, the outgoing GBA transitions are first used to generate candidate NBA transitions and the corresponding candidate NBA successor states. For each candidate successor, the induced plan node is evaluated by a lower-bound estimate. The NBA construction then prioritizes the candidate successor with the smallest lower-bound value for further expansion, rather than following the purely DFS-based expansion order used in the original construction. If several candidate successors have the same lower-bound value, the tie is broken according to their original generation order. This lower-bound-prioritized expansion preserves the on-the-fly nature of the NBA construction while biasing the search toward automaton branches that are more promising from the planning perspective. As a result, the first feasible prefix–suffix solution is more likely to have a smaller cost, which provides a higher-quality initial plan.

Let $G_\varphi = (Q_g, Q_{g,0}, \Sigma, \rightarrow_g, F_g)$ be the pruned GBA with $F_g = \{F_{g,1}, \ldots, F_{g,m}\}$. Each NBA state is represented as $q_B = (q_g, \ell)$, where $q_g \in Q_g$ is a GBA state and $\ell \in \{0, 1, \ldots, m\}$ records the current progress through the accepting transition sets. Starting from the initial GBA state, the DFS expansion visits an NBA state $(q_g, \ell)$, generates its successors from the outgoing GBA transitions of $q_g$, and updates the progress index according to the accepting transition sets carried by each GBA transition.

Based on the above process, the proposed method constructs the NBA together with a plan graph. The method expands the planning structure whenever a new NBA state or transition becomes available. Once a new NBA transition and the label of its source state are available, the corresponding subtask can be obtained and used to extend the plan graph. Therefore,

the algorithm does not need to wait until the entire NBA is constructed. A feasible solution can be returned quickly once an accepting prefix–suffix structure has been generated in the current plan graph. To connect the generated NBA structure with task-level planning, we associate NBA transition with subtasks.

**Definition 7** (Subtask)**.** Consider an NBA transition

$$e_{ij} = (q_i, \sigma_{ij}, q_j) \in \rightarrow_B,$$

where $q_i, q_j \in Q_B$ and $\sigma_{ij} \in \Sigma_B$. Let $\sigma_i^{\mathrm{sl}}$ denote the self-loop condition at the starting state $q_i$. The subtask associated with $e_{ij}$ is defined as

$$T_{ij} = (\sigma_{ij}, \sigma_i^{\mathrm{sl}}).$$

Here, $\sigma_{ij}$ describes the condition that enables the transition from $q_i$ to $q_j$, while $\sigma_i^{\mathrm{sl}}$ describes the condition that should be maintained at $q_i$ before this transition is completed.

### A. Plan Graph and Planning Node

To represent the task progress and robot execution state compactly, we construct a directed acyclic graph (DAG) called plan graph, denoted by $\mathcal{G}_P = (\mathcal{V}_P, \mathcal{E}_P)$, where $\mathcal{V}_p$ is the set of plan nodes and $\mathcal{E}_p$ is the set of directed edges among them. Unlike tree-based planning structures such as [11], we construct a plan graph to reduce memory usage. In a pure tree, different partial plans that reach the same Buchi state with the same subtask but different execution states (e.g., robot positions, completion times) would still generate separate copies of the subsequent suffix structure to preserve optimality. This duplication can become severe for complex temporal specifications. By allowing multiple parent nodes to share common anchors and successor frontiers, the proposed plan graph avoids repeated suffix expansion and alleviates potential memory explosion.

Each planning node is defined as below.

**Definition 8** (Plan node)**.** Consider the plan graph $G_P = (V_P, E_P)$, where $V_P$ is the set of plan nodes and $E_P \subseteq V_P \times V_P$ is the set of directed edges. Each plan node is denoted by $\nu_i$, and

$$V_P = \{\nu_i \mid i = 0, 1, \ldots, N_P\},$$

where $\nu_0$ is the root node and $N_P + 1$ is the number of nodes in the generated plan graph. Each plan node $\nu_i \in V_P$ is defined as

$$\nu_i = (q_{B,i}, \mathrm{Prog}_i, T_i, C_i, X_i, \Theta_i, \mathrm{LB}_i).$$

Here, $q_{B,i} \in Q_B$ is the NBA state associated with $\nu_i$, and $\mathrm{Prog}_i \in \{\mathrm{pre}, \mathrm{suf}, \mathrm{dead}\}$ denotes the current planning stage of $\nu_i$. Specifically, $\mathrm{Prog}_i = \mathrm{pre}$ indicates that the current node is in the prefix stage and $\mathrm{Prog}_i = \mathrm{suf}$ indicates that the current node is in the suffix stage. $\mathrm{Prog}_i = \mathrm{DEAD}$ indicates that the current node does not need to be further expanded.The element $T_i$ denotes the subtask. The element $C_i$ records the robot assignment.Let $\mathcal{A}(T_i) \subseteq A$ denote the set of action propositions that are required to be executed in $T_i$. Specifically, for each $a_j \in \mathcal{A}(T_i)$,$C_i(a_j) \subseteq \mathcal{R}$ denotes the set of robots assigned to execute $a_j$.

The vector

$$X_i = (x_i^1, x_i^2, \ldots, x_i^{|R|})$$

denotes the predicted robot positions, where $x_i^r$ is the predicted position of robot $r$. The vector

$$\Theta_i = (\theta_i^1, \theta_i^2, \ldots, \theta_i^{|R|})$$

denotes the predicted completion times of robots, where $\theta_i^r$ is the predicted completion time of robot $r$. Finally, $\mathrm{LB}_i$ is a lower bound estimation of any complete plan extended from $\nu_i$.

By Definition 8, a plan node records the progress in the automaton and the predicted execution state of the robot team. Specifically, $q_{B,i}$, $\mathrm{Prog}_i$, and $T_i$ describe where the current partial plan is in the NBA and which subtask is being considered. The assignment $C_i$ specifies which robots execute the current subtask, while $X_i$ and $\Theta_i$ summarize the predicted robot configuration after this subtask is completed. The lower bound $\mathrm{LB}_i$ is used to evaluate whether the current partial plan can still lead to a solution better than the current best solution.

### B. Construction of Plan Graph

The purpose of plan-graph construction is to tightly couple automaton expansion with executable task-level evaluation, so that feasible accepting plans can be identified as early as possible and subsequently used to warm-start the branch-and-bound search. Starting from the root node $\nu_0$, which is associated with the initial NBA state $q_{B,0}$ and the initial team configuration, the method initializes the predicted robot positions $X_0$ from the robots' initial states, sets the predicted completion-time vector $\Theta_0$ to zero, and begins with an empty task assignment. The NBA expansion and the plan-graph expansion are then carried out in an on-the-fly and synchronized manner. Specifically, whenever an NBA state $q_B$ generates a transition $q_B \xrightarrow{\sigma} q'_B$, the algorithm creates, or reconnects to, a compatible successor structure in the plan graph, computes a greedy synchronized assignment for the subtask induced by $\sigma$, and updates the predicted robot positions and completion times accordingly. In this way, the construction process is guided primarily by executable task assignments and their realized costs, while infeasible candidates are discarded whenever no valid synchronized assignment exists.

The resulting plan graph is therefore a shared and dynamically evolving directed structure. Compatible successor structures may be reused, reconnected, and further extended as additional admissible contexts are discovered during the expansion process, thereby reducing redundant exploration while preserving the relevant planning semantics. Whenever an accepting prefix–suffix configuration is reached, a feasible plan can be extracted online by tracing a valid predecessor chain from the accepting node back to the root. This plan is immediately recorded as the current incumbent, and it is further updated if later accepting structures yield a lower cost during the same construction phase. The best feasible solution produced by the plan graph is then provided as a warm start for the subsequent branch-and-bound stage. After the construction phase is completed, a unified graph simplification procedure

is applied to remove dead, unreachable, and locally dominated portions of the shared structure while preserving the live corridors that can still contribute to accepting solutions.

*1) Child Node Generation:* The plan graph is grown by extending a plan node $\nu_i$ along the outgoing NBA transitions of its associated state $q_{B,i}$. For each transition $e_B : q_{B,i} \xrightarrow{\sigma} q'_B$, the induced subtask is determined by the transition condition $\sigma$ together with the self-loop condition $\sigma_i^{\mathrm{sl}}$ at $q_{B,i}$, i.e., $T_j = (\sigma, \sigma_i^{\mathrm{sl}})$. The corresponding child handling then produces a successor associated with $q'_B$ by extending the partial plan rooted at $\nu_i$. Importantly, this successor is not necessarily a newly instantiated node. Depending on the current shared graph structure, the expansion may create a new plan node, reconnect to a compatible existing anchor, or reuse and further grow a previously generated successor structure. Compatibility is determined by the stage information together with the matched transition semantics carried by the shared structure. In all cases, the resulting successor represents the extended partial plan together with the updated predicted robot states and the executable cost induced by this extension.

The successor state is evaluated through a greedy synchronized task-assignment procedure. Let $\mathcal{A}(T_j) \subseteq A$ denote the set of action propositions required by $T_j$, and let $c(a_\ell) = (\tau_\ell, n_\ell, l_\ell)$ denote the execution requirement of $a_\ell \in \mathcal{A}(T_j)$, where $\tau_\ell$, $n_\ell$, and $l_\ell$ are the required robot type, required number of robots, and target region, respectively. For each candidate robot $r \in \mathcal{R}_{\tau_\ell}$, the predicted arrival time to $l_\ell$ is estimated by

$$\hat{\theta}_\ell^r = \theta_i^r + \frac{d(x_i^r, l_\ell)}{v_r},$$

where $x_i^r$ and $\theta_i^r$ are the predicted position and predicted completion time of robot $r$ at $\nu_i$. The $n_\ell$ robots with the smallest predicted arrival times are selected, synchronized at the maximum of their predicted arrival times, and then used to update the predicted positions and completion times in the successor node. If fewer than $n_\ell$ eligible robots are available, the candidate is declared infeasible and discarded. The planning stage of the successor is updated according to the prefix–suffix progress of the current path: the expansion remains in the prefix stage until an accepting NBA state is reached, switches to the suffix stage when such a state is crossed from the prefix stage, and, once an accepting prefix–suffix execution is completed, immediately yields a feasible incumbent online. In this sense, child generation is guided by executable task assignments and their realized costs, while the detailed priority policy among generated successors is deferred to the traversal rules described next.

*2) Heuristic Scoring of Büchi State:* To bias on-the-fly Büchi expansion toward successors with lower residual obligation pressure, we assign a lightweight heuristic score to each parent-transition pair $(p, \tau)$, where $p$ is the current plan node and $\tau$ is a candidate Büchi transition. The score is computed from the remaining must-eventually propositions together with the current execution state carried by $p$.

we recursively extract a must-eventually set from the abstract syntax tree (AST) of the task formula. For the global LTL formula $\varphi$, we define a recursive mapping $\mathsf{ME}(\cdot)$ on the subformulas in the AST of $\varphi$. Here, $\psi$ denotes an arbitrary subformula of $\varphi$. For such a subformula $\psi$, $\mathsf{ME}(\psi) \subseteq A$ denotes the set of positive action propositions that are extracted as unavoidable when $\psi$ is recursively evaluated. The must-eventually set of the original task formula is then given by $M = \mathsf{ME}(\varphi)$.The recursive rules for computing $\mathsf{ME}(\psi)$ are defined as follows.

1) The constants $\top$ and $\bot$ do not introduce any positive action proposition that must be completed. That is, if $\psi = \top$ or $\psi = \bot$, then $\mathsf{ME}(\psi) = \emptyset$.
2) Only positive action propositions are included in the lower-bound estimate. A negated formula imposes a prohibition rather than a positive action proposition that must be completed, i.e. If $\psi = \neg\psi_1$, then $\mathsf{ME}(\psi) = \emptyset$ and if $\psi = a \in \mathcal{A}$, then $\mathsf{ME}(\psi) = a$
3) If $\psi = \psi_1 \wedge \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1) \cup \mathsf{ME}(\psi_2)$. Since both subformulas must be satisfied, the action propositions are collected from both sides.
4) If $\psi = \psi_1 \vee \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1) \cap \mathsf{ME}(\psi_2)$. Since either branch may be selected, only action propositions that are unavoidable in both branches are kept.
5) If $\psi = \bigcirc\psi_1$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1)$. The next operator shifts the satisfaction time but does not change the unavoidable positive action propositions.
6) If $\psi = \psi_1 \,\mathcal{U}\, \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_2)$. For an until formula, satisfaction requires the right-hand subformula to eventually hold.
7) If $\psi = \psi_1 \,\mathcal{V}\, \psi_2$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_2)$. In the implemented rule for the release operator, the positive action requirements from the right-hand subformula are extracted.
8) If $\psi = \Box\psi_1$ or $\psi = \Diamond\psi_1$, then $\mathsf{ME}(\psi) = \mathsf{ME}(\psi_1)$. The always and eventually operators preserve the positive action propositions appearing in the operand.

Based on the above recursive rules, the must-eventually set of the global task formula is obtained as $M$. The set $M$ should not be interpreted as the set of action propositions that must be satisfied immediately at the current plan node. Instead, it provides a formula-level summary of unavoidable positive action propositions. This summary is used to estimate the minimum remaining effort when the future subtasks have not been fully generated during the incremental NBA construction.

Consider a source Büchi state $q_B$, an outgoing transition $\tau : q_B \xrightarrow{\sigma} q'_B$, and an active plan node $p$ currently attached to $q_B$. Let $J(p)$ denote the current cost of $p$, and let $M \subseteq A$ be the set of must-eventually atomic propositions extracted from the task formula. For the candidate expansion $(p, \tau)$, we first determine the set of residual obligations that would remain after taking $\tau$. To this end, we construct a shared progress summary for the target state $q'_B$: if $q'_B$ already contains compatible active anchors, i.e., anchors with the same stage and the same positive/negative transition signature as $\tau$, their progress summaries are combined by intersection; otherwise, the summary carried by $p$ is used directly. After incorporating the positive literals asserted by $\tau$, we obtain the residual must-eventually set

$$R(p, \tau) = M \setminus A^+(p, \tau),$$

where $A^+(p,\tau)$ denotes the updated progress summary after applying $\tau$.

For each residual atomic proposition $a \in R(p,\tau)$, let

$$c(a) = (\text{type}(a),\, n(a),\, l(a))$$

denote its required robot type, multiplicity, and target region. Using the current execution state stored at $p$, we estimate a relaxed lower bound for satisfying $a$ alone. Specifically, for every robot $r$ of the required type, we compute its arrival time

$$t_r(p,a) = \mathrm{ET}_p(r) + \frac{d(\mathrm{EP}_p(r),\, l(a))}{v_r},$$

where $\mathrm{ET}_p(r)$ and $\mathrm{EP}_p(r)$ are the current completion time and position of robot $r$, respectively. Sorting these arrival times in nondecreasing order,

$$t_{(1)}(p,a) \le t_{(2)}(p,a) \le \cdots,$$

the relaxed lower bound for proposition $a$ is defined as

$$\underline{t}(p,a) = t_{(n(a))}(p,a).$$

This estimate ignores the coupling constraint that the same robot cannot simultaneously execute two different future tasks; therefore, it is a relaxed bound on satisfying $a$ individually.

The residual pressure induced by $(p,\tau)$ is then defined by the bottleneck over all remaining obligations:

$$H(p,\tau) = \begin{cases} \max\limits_{a \in R(p,\tau)} \underline{t}(p,a), & R(p,\tau) \neq \emptyset, \\ J(p), & R(p,\tau) = \emptyset. \end{cases}$$

The heuristic score of the candidate expansion $(p,\tau)$ is given by

$$S(p,\tau) = \max\big(J(p),\, H(p,\tau)\big).$$

Hence, $S(p,\tau)$ combines the cost already accumulated along the current partial plan with a relaxed estimate of the remaining must-eventually obligations.

Finally, a candidate successor Büchi state $q'_B$ may be reachable from multiple active plan nodes attached to $q_B$. In that case, the score retained for $q'_B$ is the best one among all candidate expansions:

$$\mathrm{Score}(q'_B) = \min_{p \in \mathcal{P}(q_B)} S(p,\tau),$$

where $\mathcal{P}(q_B)$ is the set of active plan nodes associated with $q_B$. Therefore, the implementation first evaluates each parent-specific expansion by a relaxed obligation-based estimate, and then keeps the minimum score as the priority signal for the pending Büchi successor.

### C. Traversal Rules

The current implementation constructs a shared directed plan graph and applies the following traversal rules.

1) **Local expansion order.** For a fixed Büchi state, the currently active parent anchors are processed in nondecreasing realized cost. This ordering is only a local exploration preference for that state; it is not a global best-first guarantee over the full graph.
2) **Local candidate precheck.** Before expanding a candidate transition from a given parent context, the implementation may apply a limited local ancestor-repeat precheck on selected entry paths. This precheck filters some locally redundant candidates, but it does not guarantee that the retained shared graph is acyclic.
3) **Dynamic reuse, reconsideration, and fresh generation.** For each outgoing Büchi transition, the child phase is derived from the parent phase and current progress context. The planner then seeks compatible previously constructed structure at the destination state, where compatibility is defined by the destination Büchi state, the induced phase, and the positive/negative transition-label signature. In this step, the implementation may reconnect to an already active compatible node or reconsider a previously suspended branch of the same signature when a lower-cost incoming parent context becomes available. If neither option is suitable, a new child is generated, provided that the induced synchronized task assignment is feasible.
4) **Cost-based representative-state update.** Whenever an existing compatible node is reused, its state is recomputed under the new parent context. Each shared node maintains a representative predecessor rather than an immutable first predecessor. This representative state is updated only when the recomputation yields a strictly smaller realized cost, and any accepted improvement is propagated forward through the reused shared structure.
5) **Late-arrival recovery.** If a newly created node reaches a state whose outgoing structure has already been explored, the node is retained rather than discarded. The implementation then attempts to reconnect it to compatible already existing successors at that state and, when useful, may extend this recovery over newly recovered local successors. Hence, successor sharing is determined dynamically by the evolving graph structure and lower-cost reconnections, rather than by a fixed static rule.
6) **Inactive-branch treatment.** The implementation distinguishes between discarded failed branches and retained dormant stopped branches. In the former case, the node is removed only from the current local graph context and marked inactive. In the latter case, the stopped node is excluded from ordinary forward traversal but may still be reconsidered by the dynamic reuse policy above. In particular, inactive handling is not implemented as an eager recursive backward release of the entire ancestor chain.
7) **Post-construction simplification.** After graph generation, the implementation performs a distinct simplification pass. First, it identifies the live corridor by tracing backward from accepting nodes and removes edges or subgraphs outside that corridor. Second, it cleans up structure associated with hard-dead states and retained stopped remnants that no longer support any live accepting corridor. Third, among sibling successors of the same parent, it prunes locally dominated alternatives when one child subgraph covers another with no larger realized cost. In the current implementation, this post-

construction pass is the main mechanism for removing redundant retained structure.

### D. Final Post-Construction Pruning

After the shared plan graph has been constructed, we apply a final simplification pass from the root. In the implementation, this pass is invoked once as `simplify_plan_tree`($pRoot$, `HARD_DEAD_BSTATE_ADDRS`), where $\mathcal{D}_{\text{hard}}$ denotes the supplied set of hard-dead specification-state pointers. The pass removes child links that cannot support a root-to-accepting continuation, while respecting that a plan node may be shared by multiple parents.

The pass first marks live nodes. Starting from the root, it traverses active child links to identify the root-reachable subgraph. Reachable accepting nodes initialize the live set, and liveness is then propagated backward through parent links. Thus, a live node is root-reachable and lies on an active path that can reach at least one reachable accepting node. The retention condition is that a non-accepting node must keep at least one active child link into the live set, whereas an accepting node may terminate a continuation and is retained even as a leaf.

Pruning is performed in an edge-first manner. For each active child link, the link is removed if the child is not in the live set, if the child is non-accepting and has no live successor, if the child's specification-state pointer belongs to $\mathcal{D}_{\text{hard}}$, if the child carries the CUT flag, or if recursive simplification returns false. The same pruning is applied recursively below retained children; after descendants are pruned, a non-accepting child that no longer satisfies the live-successor condition is also removed. Because the graph is shared, removing a link does not immediately retire or free the child node itself. A child node is retired only after all of its incoming parent links have been removed, preventing one parent from freeing a subgraph that is still referenced elsewhere.

The pass also applies local sibling dominance pruning. For each parent, children are compared only with siblings having the same local signature

$$\sigma(v) = \big(q(v),\, F(v),\, P^{+}(v),\, P^{-}(v),\, \bar{P}^{+}(v)\big),$$

where $q(v)$ is the specification-state pointer, $F(v)$ is the node flag, and $P^{+}(v)$, $P^{-}(v)$, and $\bar{P}^{+}(v)$ are the positive, negative, and already-satisfied proposition tuples. Within a common parent and signature class, a candidate child may dominate another child only if a recursive structural coverage test succeeds and the candidate's stored node cost is no larger, up to the numerical tolerance used in implementation. When domination holds, only the dominated sibling link is removed; the child node is retired only if it has no remaining parents. The resulting graph is a compacted post-construction representation, with no claim of global minimality or optimality, that retains accepting leaves and surviving branches under the stated tests.

*1) Plan Extraction:*

*2) Plan Extraction:*

## VII. BRANCH-AND-BOUND OPTIMIZATION

After the plan graph is constructed, the remaining problem is to choose concrete robot assignments for the synchronized subtasks encoded by the graph. We solve this discrete assignment problem with branch-and-bound (BnB), warm-started when an initial feasible solution is available. The optimization criterion is the makespan objective,

$$J_{\text{final}} = \max_{r_i \in \mathcal{R}} ET_i, \tag{3}$$

where $ET_i$ is the completion time of robot $r_i$. The incumbent cost $J^{\star}$ is initialized from a plan-graph solution or a greedy seed when available, and BnB then attempts to improve this incumbent. If the frontier is exhausted without termination by a runtime, memory, or open-list guard, the incumbent is optimal within the generated plan-graph assignment space; otherwise, the algorithm returns the best feasible incumbent found.

### A. Node Expansion

We store each BnB node as

$$\mu = (v, \Pi, \mathbf{EP}, \mathbf{ET}, d, g, F), \tag{4}$$

where $v$ is the current plan node, $\Pi$ stores the per-robot task sequences, $\mathbf{EP}$ and $\mathbf{ET}$ are the current robot positions and completion times, $d$ is the search depth, $g$ is the current makespan, and $F$ is the node priority value.

Expanding $\mu$ considers each child plan node $v^{+}$ of $v$. For the synchronized subtask associated with $v^{+}$, the algorithm enumerates all feasible robot combinations satisfying the required atomic propositions. Atomic propositions are processed in descending cardinality so that higher-cardinality requirements are assigned first. For an atomic proposition $a$ with required robot type $\tau_a$, target region $\mathcal{X}_a$, and cardinality $n_a$, only robots of type $\tau_a$ are eligible, and each robot can be used at most once within the same synchronized subtask.

For a complete synchronized assignment $\sigma$, the selected robots travel from their current positions to the assigned regions. If robot $r_i$ is assigned to proposition $a$, its arrival-time estimate is

$$\hat{T}_i(a) = ET_i + T(EP_i, \mathcal{X}_a), \tag{5}$$

where $T(\cdot,\cdot)$ denotes the travel-time estimate. The synchronized subtask finishes at

$$T_\sigma = \max_{r_i \in \mathrm{dom}(\sigma)} \hat{T}_i(\sigma(r_i)). \tag{6}$$

All selected robots are synchronized to this finish time, while unselected robots inherit their parent states. The child cost is therefore

$$g^{+} = \max\{g, T_\sigma\} = \max_{r_i \in \mathcal{R}} ET_i^{+}. \tag{7}$$

Assignments with $g^{+} \geq J^{\star}$ can be pruned immediately.

### B. Branching Rule

The search maintains an open list ordered in best-first fashion. For each generated child assignment state, the priority value is

$$F(\mu) = \max\{g(\mu), LB(\mu)\}, \tag{8}$$

where $LB(\mu)$ is the admissible lower-bound estimate for that generated state. The open list therefore prioritizes the smallest lower estimate of the attainable makespan after accounting for the current cost. The heap key uses $F$ as the primary key and a concurrency score as a secondary tie-breaker. Generated nodes are pushed to the open list only if $F(\mu) < J^\star$.

### C. Lower Bound

The lower bound provides a relaxed estimate of the earliest completion time required by future atomic propositions reachable from the current plan node. Let $\mathcal{A}_{\mathrm{rem}}(v)$ denote the set of remaining required atomic propositions that have not yet been satisfied, augmented with requirements shared by relevant outgoing continuations. If this set is empty, then

$$LB(\mu) = g(\mu). \tag{9}$$

For each required proposition $a \in \mathcal{A}_{\mathrm{rem}}(v)$, with type $\tau_a$, target region $\mathcal{X}_a$, and cardinality $n_a$, the bound considers all type-compatible robots in the current assignment state:

$$\mathcal{C}_a(\mu) = \{r_i \in \mathcal{R} \mid \mathrm{type}(r_i) = \tau_a\}. \tag{10}$$

For each candidate robot,

$$h_i(a) = ET_i + T(EP_i, \mathcal{X}_a). \tag{11}$$

After sorting these values in nondecreasing order, the relaxed completion estimate of $a$ is the $n_a$-th smallest arrival time,

$$\ell(a, \mu) = h_{(n_a)}(a). \tag{12}$$

The node lower bound is

$$LB(\mu) = \max\Big(g(\mu), \max_{a \in \mathcal{A}_{\mathrm{rem}}(v)} \ell(a, \mu)\Big). \tag{13}$$

This relaxation ignores downstream ordering and resource coupling and does not update robot states while evaluating multiple future propositions. If fewer than $n_a$ type-compatible robots exist, or if no feasible target region exists, the bound is set to $+\infty$.

### D. Upper Bound

After expanding a child plan node, BnB retains the best candidate assignment generated for that child and attempts a feasible rollout from it. The rollout follows the plan graph by repeatedly selecting the child preferred by the rollout heuristic and then applying inherited plan assignments or a greedy completion policy.

A rollout is accepted only if it reaches a terminal accepting plan node and all synchronized assignments along the rollout are valid. It is rejected if it encounters a cycle, an invalid task assignment, a configured step, time, memory, or open-list guard, or a partial cost $J \geq J^\star$. When a valid rollout produces a makespan $J_{\mathrm{roll}} < J^\star$, the incumbent is updated:

$$J^\star \leftarrow J_{\mathrm{roll}}, \tag{14}$$

together with the corresponding assignment plan and plan-node path. The rollout is used only to improve the incumbent; it is not an optimality certificate.

### E. Pruning

A node $\mu$ is pruned if one of the following conditions holds. First, if $g(\mu) \geq \bar{J}$, then even the current partial assignment cannot improve the incumbent. Second, if $F(\mu) \geq \bar{J}$, where $F(\mu)$ is defined in (**??**), then no complete assignment rooted at $\mu$ can improve the incumbent under the admissible lower bound. Third, if the type or cardinality requirement of a remaining subtask cannot be satisfied, the node is removed as infeasible.

## VIII. Complexity Analysis

This section analyzes the computational complexity of the proposed framework by decomposing it into three coupled stages: (i) pruning of the generalized Büchi automaton (GBA), (ii) planning and lower-bound-based pruning during the construction of the nondeterministic Büchi automaton (NBA), and (iii) branch-and-bound search for synchronized task allocation. In contrast to approaches that first construct the full automaton and then perform planning, the proposed method pushes feasibility checks, structural simplifications, lower-bound evaluation, and assignment pruning as early as possible. As a result, although the worst-case combinatorial nature of the problem is not eliminated, the practical runtime and the latency to the first feasible solution are significantly reduced.

For clarity, let

- $n_G = |S_G|$ and $e_G = |E_G|$ denote the number of states and transitions of the GBA, respectively;
- $d_G = \max_{s \in S_G} \deg^+(s)$ denote the maximum out-degree of the GBA;
- $n_B = |S_B|$ and $e_B = |E_B|$ denote the number of states and transitions of the NBA, respectively;
- $R$ denote the total number of robots, and $R_\tau$ denote the number of robots of type $\tau$;
- $m$ denote the number of atomic propositions (APs) that are simultaneously processed at a planning node;
- $k_i$ denote the number of robots required by the $i$-th AP;
- $V_W$ and $E_W$ denote the number of nodes and edges in the workspace graph, respectively;
- $N_P$ denote the number of surviving `PlanNodes` after planning-aware pruning during NBA generation.

Furthermore, if a shortest-path query in the workspace graph is not available in cache, its computational cost is

$$T_{\mathrm{sp}} = O((V_W + E_W) \log V_W). \tag{15}$$

When the cache is hit, this cost can be regarded as $O(1)$.

### A. Complexity of GBA Pruning

The GBA-related operations consist of candidate transition generation, feasibility pruning, SCC-based pruning, transition dominance elimination, state merging, and the proposed ST pruning.

*1) Feasibility pruning:* During the construction of outgoing GBA transitions, every candidate conjunction of positive and negative literals is checked for logical feasibility before it is propagated to later stages. Let $C_s$ denote the number of candidate transition combinations generated at a GBA state $s \in S_G$, and let $T_f$ denote the cost of one feasibility test. Then the total complexity of feasibility pruning is

$$O\left(\sum_{s\in S_G} C_s T_f\right). \tag{16}$$

This step is not necessarily the dominant term by itself. However, its main contribution is that it removes unsatisfiable transition labels at the earliest possible stage, thereby reducing the effective number of transitions that are subsequently processed by state generation, transition comparison, and state merging. Hence, feasibility pruning yields a multiplicative reduction in the size of all downstream computations.

*2) ST pruning:* The proposed ST pruning removes redundant direct transitions of the form $v_1 \to v_3$ when they are subsumed by a two-hop path $v_1 \to v_2 \to v_3$ under the same logical and acceptance conditions. Using $d_G$ as an upper bound on the out-degree, this procedure has complexity

$$O(n_G d_G^3). \tag{17}$$

*3) Overall complexity of GBA pruning:* Combining the above terms, the total complexity of the GBA pruning stage is

$$O\left(\sum_{s\in S_G} C_s T_f\right)+O(n_G+e_G)+O(e_G d_G)+O(n_G^2 d_G^2)+O(n_G d_G^3). \tag{18}$$

Retaining only the dominant terms yields

$$O\left(\sum_{s\in S_G} C_s T_f + n_G^2 d_G^2 + n_G d_G^3\right). \tag{19}$$

It is important to note that the proposed feasibility pruning and ST pruning do not remove the exponential source of automaton construction in the worst case. If most candidate transitions are feasible and no ST redundancy exists, the asymptotic worst-case order remains unchanged. Nevertheless, in practical implementations these pruning operations substantially reduce the effective GBA size before later planning stages begin.

### B. Complexity of Planning and Lower-Bound Pruning During NBA Generation

A key feature of the proposed approach is that planning is embedded directly into NBA construction. Instead of first generating the complete NBA and then performing planning, every automaton expansion is immediately filtered by synchronized task assignment and lower-bound pruning. As a result, only branches that are both automaton-consistent and planning-promising are preserved.

*1) Greedy synchronized assignment:* For a planning node containing $m$ APs, the greedy synchronized assignment scans the robots of the required type for each AP, computes their arrival times, and sorts the candidates. If the $i$-th AP is associated with robot type $\tau_i$, then the complexity is

$$O\left(\sum_{i=1}^{m} R_{\tau_i} \log R_{\tau_i}\right). \tag{20}$$

A coarse upper bound is

$$O(mR \log R). \tag{21}$$

If the required shortest-path distances are not cached, this term must additionally include the workspace path-query cost $T_{\mathrm{sp}}$.

*2) Lower-bound estimation:* The lower-bound function evaluates the earliest achievable execution times of the remaining APs by tracing back along the ancestor chain and accounting for robot-type reuse and inter-region travel costs. If the planning depth is $L$, and each level contains at most $m$ APs, then an upper bound on the lower-bound evaluation cost is

$$O(m^2 L). \tag{22}$$

Since repeated visits to the same NBA state under the same planning stage are excluded along a path, the effective planning depth is typically bounded linearly by the NBA size, i.e.,

$$L = O(n_B). \tag{23}$$

Therefore, the lower-bound evaluation can be upper bounded by

$$O(m^2 n_B), \tag{24}$$

again excluding cache misses in shortest-path queries.

*3) Overall complexity during NBA generation:* Let $N_P$ denote the number of surviving planning nodes after synchronized assignment checks and lower-bound pruning. Then the total complexity of this stage can be expressed as

$$O\left(N_P \left(mR \log R + m^2 n_B\right)\right), \tag{25}$$

plus the cost of uncached shortest-path evaluations.

This expression highlights that the main benefit of the proposed online planning strategy is not necessarily a better worst-case asymptotic order than a "construct-first-plan-later" strategy, but a substantial reduction in $N_P$ in practice. Indeed, planning feasibility and lower-bound information are used at the moment of automaton expansion, thereby preventing many automaton-valid but planning-hopeless branches from entering deeper search.

*4) Theoretical versus practical effect:* From a theoretical viewpoint, constructing the full NBA first and planning afterward may still lead to the same optimal solution as the proposed integrated approach. However, the integrated construction is significantly more efficient in practice because:

1) planning infeasibility is detected before the corresponding automaton branch grows deeper;
2) a feasible solution is typically found earlier, which provides a tighter global upper bound;
3) the tighter upper bound strengthens subsequent lower-bound pruning.

Therefore, embedding planning into NBA generation does not necessarily alter the worst-case exponential nature of temporal planning, but it substantially reduces the time required to obtain the first feasible solution.

### C. Complexity of the Branch-and-Bound Assignment Search

The most significant source of combinatorial complexity in the entire framework arises in the synchronized multi-robot assignment stage. This stage is handled by the branch-and-bound procedure in `branch_bound_A`.

*1) Assignment enumeration complexity:* Consider a planning node with $m$ APs. Suppose that for the $i$-th AP, $k_i$ robots must be selected from $r_i$ candidate robots. Ignoring cross-task conflicts for an upper-bound estimate, the number of possible assignments satisfies

$$M \leq \prod_{i=1}^{m} \binom{r_i}{k_i}. \tag{26}$$

Although the actual number of feasible assignments is smaller due to the constraint that robots cannot be reused across different APs, the same combinatorial order remains.

For each complete assignment, the algorithm clones the assignment state, updates robot completion times and positions, and records the selected tasks. Let $C_{\mathrm{gen}}$ denote the cost of generating one full assignment. Then

$$C_{\mathrm{gen}} = O\left(R + \sum_{i=1}^{m} k_i\right), \tag{27}$$

and the total complexity of assignment enumeration at one planning node is

$$O\left(\prod_{i=1}^{m} \binom{r_i}{k_i} \left(R + \sum_{i=1}^{m} k_i\right)\right). \tag{28}$$

This is the primary worst-case complexity driver of the whole algorithm.

*2) Lower-bound complexity in branch-and-bound:* For every assignment that survives initial filtering, a lower-bound function is evaluated in order to compute the $f$-value used by the A*-like search. Let $N_{\mathrm{rem}}$ denote the size of the remaining planning subtree below the current node. Then one lower-bound evaluation has complexity

$$O(N_{\mathrm{rem}} \cdot m \cdot R_{\max} \log R_{\max}), \tag{29}$$

where

$$R_{\max} = \max_{\tau} R_{\tau}. \tag{30}$$

*3) Upper-bound complexity:* The fast upper-bound routine performs a rollout along one descendant path until a leaf is reached. If the rollout length is $L_{\mathrm{roll}}$, then its complexity is

$$O(L_{\mathrm{roll}} C_{\mathrm{roll}}), \tag{31}$$

where $C_{\mathrm{roll}}$ denotes the cost of one rollout step. Since this routine does not enumerate all assignments, it is usually much cheaper than the full combinatorial branching. Its main purpose is to obtain a tighter global upper bound as early as possible.

*4) Overall complexity of branch-and-bound:* Let $N_A$ denote the number of assignment states that are actually popped from the open list during the A*-style branch-and-bound search. Then the total complexity can be expressed as

$$O\left(N_A\left[M\left(R + \sum_i k_i\right) + M N_{\mathrm{rem}} m R_{\max} \log R_{\max} + L_{\mathrm{roll}} C_{\mathrm{roll}}\right]\right). \tag{32}$$

Retaining only the dominant term yields

$$O(N_A \cdot M \cdot N_{\mathrm{rem}} \cdot m R_{\max} \log R_{\max}), \quad M = \prod_{i=1}^{m} \binom{r_i}{k_i}. \tag{33}$$

Hence, the worst-case complexity of the branch-and-bound stage remains combinatorial.

*5) Role of upper and lower bounds:* It is crucial to emphasize that branch-and-bound does not remove the combinatorial worst-case nature of synchronized assignment. If all assignments remain feasible and the lower bound is loose, the search may approach exhaustive enumeration. However, in practice, its efficiency depends strongly on:

1) how early a good feasible assignment is found, which tightens the global upper bound;
2) how tight the lower bound is, which allows poor branches to be pruned before full expansion.

Consequently, the practical efficiency of `branch_bound_A` is determined far more by the number of actually expanded assignment states $N_A$ than by the nominal size of the combinatorial search space.

### D. Overall Effect of Early Pruning

The proposed framework benefits from pruning at three different levels:

1) GBA-level pruning eliminates logically infeasible and structurally redundant automaton branches;
2) NBA-level planning-aware pruning removes automaton-valid but execution-hopeless branches before they grow deeper;
3) assignment-level branch-and-bound reduces the number of explored synchronized allocations using upper and lower bounds.

Let $\rho_G, \rho_P, \rho_A \in [0, 1]$ denote the effective survival ratios induced by these three pruning layers, respectively. If $N_0$ denotes the size of the unpruned search space, then the effective search space that reaches the deepest assignment stage can be approximated as

$$N_0 \rho_G \rho_P \rho_A. \tag{34}$$

Therefore, earlier pruning has a multiplicative effect on the total computational effort.

This also explains the empirical observation that pruning already at the GBA level, as well as embedding planning and lower-bound pruning into NBA generation, substantially accelerates the discovery of the first feasible solution. The reason is not necessarily a change in the worst-case asymptotic order, but rather that fewer invalid or low-quality branches are expanded in the early stages and a tighter global upper bound is obtained sooner.

### E. Summary

In summary, the complexity of the proposed framework can be characterized as follows:

$$\text{GBA pruning: } O\left(\sum_{s\in S_G} C_s T_f + n_G^2 d_G^2 + n_G d_G^3\right), \tag{35}$$

$$\text{NBA generation with planning: } O\big(N_P\left(mR\log R + m^2 n_B\right)\big), \tag{36}$$

$$\text{branch-and-bound assignment: } O(N_A \cdot M \cdot N_{\text{rem}} \cdot mR_{\max}\log R_{\max}), \tag{37}$$

where

$$M = \prod_{i=1}^{m} \binom{r_i}{k_i}. \tag{38}$$

Thus, from a worst-case perspective, the dominant source of complexity remains the combinatorial synchronized assignment over multiple robots. Nevertheless, by introducing feasibility and ST pruning at the GBA stage, embedding planning and lower-bound pruning into NBA generation, and finally applying branch-and-bound in the assignment stage, the search space is progressively compressed at increasingly earlier levels. This significantly improves the time to the first feasible solution and the practical runtime of the algorithm.

## IX. THEORETICAL ANALYSIS

*Lemma* IX.1. *If there exists a word $w$ accepted by $B_\varphi$, then one can find a word $\tilde{w}$ accepted by $B_\varphi^-$ that is either equivalent to $w$ or incurs a smaller required cost. That is, pruning does not affect completeness.*

*Proof.* Our framework uses two complementary pruning mechanisms: (i) GBA pruning before NBA construction, and (ii) pruning during initial-solution generation. For the first mechanism, pruning on the GBA mainly targets infeasible transitions and decomposition-preserving transitions. Crucially, the GBA-to-NBA transformation does not convert an infeasible GBA transition into a feasible NBA transition. Therefore, removing infeasible transitions during GBA construction cannot eliminate any feasible solution in the resulting NBA. By Lemma 1, decomposition-preserving transitions in the GBA are exactly the decomposable transitions in the NBA. Hence, it is sufficient to prove correctness only for pruning decomposable transitions in the NBA.Consider an accepted word $w = \cdots\{\sigma_n\}\cdots$ in $B_\varphi$, with an accepting run$\rho = \cdots q_i q_j \cdots$, where $q_j = \delta(q_i, \sigma_n)$.If $\rho$ contains no decomposable transition, then every edge of $\rho$ remains in $B_\varphi^-$. Hence $\rho$ is still an accepting run in $B_\varphi^-$, and we can choose an equivalent accepted word $w' = w$. Suppose the transition from $q_i$ to $q_j$ is removed because it is decomposable. By Definition of decompse transition, there exist a state $q_k$ and labels $\sigma_{ik}, \sigma_{kj}$ such that$q_k = \delta(q_i, \sigma_{ik})$, $q_j = \delta(q_k, \sigma_{kj})$, and $\sigma_{ik} \cup \sigma_{kj} = \sigma_n$.Then we insert one additional time step in place of $\{\sigma_n\}$ to construct an equivalent word $w' = \cdots\{\sigma_{ik}\}\{\sigma_{kj}\}\cdots$, with associated run $\rho' = \cdots q_i q_k q_j \cdots$. If the two new transitions are not removed in $B_\varphi^-$, this reduces to Case 1, so $w'$ is accepted by $B_\varphi^-$. Otherwise, if one of them is still removed as decomposable, we repeat the same insertion procedure. By Definition of decompse transition, this refinement process cannot continue indefinitely; after finitely many insertions, we obtain a run with no decomposable transition. Therefore, there exists an equivalent word accepted by $B_\varphi^-$.

For the second pruning mechanism during initial-solution generation, we adopt a branch-and-bound style rule: any branch whose best possible required cost cannot be lower than that of another candidate word is pruned. Hence, every pruned branch is dominated by at least one better word and cannot contain a cost-improving solution. For the second pruning mechanism during initial-solution generation, we adopt a branch-and-bound style rule: any branch whose best possible required cost cannot be lower than that of another candidate word is pruned. Hence, every pruned branch is dominated by at least one better word and cannot contain a cost-improving solution.

□

*Lemma* IX.2. *In the LTL2BA algorithm, offline optimization of the NBA does not affect feasible solutions.*

*Proof.* Offline optimization of the NBA mainly removes dead ends and unnecessary loops; therefore, it does not affect feasible solutions. □

*Lemma* IX.3. *Our initial algorithm is complete, i.e., if a feasible solution exists, the initial algorithm is guaranteed to find it.*

*Proof.* To prove the completeness of the proposed initial solution algorithm, we need to show that it can always find a feasible plan if one exists. The algorithm incrementally constructs a planning tree along with the generation of the NBA. Starting from the root node, it expands nodes by following all valid transitions of the NBA and generates corresponding plan nodes. Therefore, before applying any pruning strategy, the planning tree systematically explores all feasible paths induced by the NBA. Assume that there exists a feasible plan

$$\Pi^{\text{fea}} = s_0 s_1 \dots s_n$$

that satisfies the specification, where the corresponding sequence of NBA states is

$$(q_0, f_0), (q_1, f_1), \dots, (q_n, f_n).$$

According to the construction of the planning graph, for each transition $(q_i, f_i) \to (q_{i+1}, f_{i+1})$, the algorithm will generate the corresponding successor node during expansion. Hence, there exists a sequence of nodes in the planning tree that corresponds to $\Pi^{\text{fea}}$, and the node corresponding to $(q_n, f_n)$ will be reached if the path is fully expanded.

The algorithm employs a greedy strategy to obtain an initial solution, which provides an upper bound on the solution cost. Furthermore, a lower-bound estimation is used to prune branches whose lower bound exceeds the current best solution (branch-and-bound mechanism). Importantly, for any feasible plan $\Pi^{\text{fea}}$, the corresponding branch will only be pruned if its lower bound is greater than the current upper bound. In such a case, $\Pi^{\text{fea}}$ cannot yield a better solution than the current best

one, and thus pruning does not affect the completeness with respect to finding feasible solutions. On the other hand, if the lower bound of $\Pi^{\text{fea}}$ does not exceed the current upper bound, the corresponding branch will not be pruned, and the algorithm will continue expanding it until the full plan is constructed. Therefore, every feasible plan that could potentially improve or match the current best solution will be fully explored, and any feasible plan will either be found or safely pruned without affecting completeness. Consequently, the proposed initial solution algorithm is complete. □

*Lemma* IX.4. *All solutions $J^\star$ obtained by the branch-and-bound algorithm satisfy the constraints of the planning tree.*

*Proof.* To prove the correctness of the branch-and-bound optimization, it suffices to show that any solution obtained by the upper-bound method is feasible, and that the search procedure does not eliminate any optimal solution.

The current best solution $J^\star$ is obtained from a node in the planning tree using the upper-bound computation. Since the planning tree is constructed based on the NBA, every node in the tree corresponds to a valid sequence of automaton states and thus satisfies the specification. Therefore, any solution generated by the upper-bound method is guaranteed to be feasible.

For any node $\nu$ selected during the search, its remaining subproblems are expanded strictly following the transitions of the NBA and the structure of the planning tree. Hence, all generated plans remain consistent with the automaton constraints.

Furthermore, the branch-and-bound mechanism prunes a node $\nu$ only when its lower bound exceeds the current best solution $J^\star$. In this case, any solution derived from $\nu$ cannot improve upon $J^\star$, and thus pruning does not affect the discovery of the optimal solution.

Since the search is conducted over the entire planning tree generated from the initial solution procedure, and pruning only removes suboptimal branches, all feasible solutions are either explored or safely discarded. Therefore, the branch-and-bound optimization is complete and guarantees to find an optimal solution. □

*Lemma* IX.5. *Given sufficient time, our algorithm is guaranteed to obtain the optimal solution.*

*Proof.* To prove the optimality of the proposed algorithm, we need to show that the solution returned by the branch-and-bound procedure is globally optimal. First, the planning tree constructed from the NBA captures all feasible plans that satisfy the specification. Therefore, the search space of the algorithm includes all possible feasible solutions. Second, the algorithm maintains an upper bound $J^\star$, which corresponds to the cost of the best feasible solution found so far. Since this solution is generated from the planning graph, it is guaranteed to satisfy all constraints and is therefore feasible. Third, the algorithm prunes a node $\nu$ only when its lower bound $LB(\nu)$ exceeds the current upper bound $J^\star$. For any node that can lead to an optimal solution with cost $J^{\text{opt}}$, it must hold that

$$LB(\nu) \leq J^{\text{opt}} \leq J^\star.$$

Thus, such a node will never be pruned before the optimal solution is discovered.

Consequently, the branch corresponding to the optimal solution remains in the search space and will eventually be explored. Once the optimal solution is found, the upper bound is updated to $J^{\text{opt}}$, and no better solution can exist.

Therefore, the algorithm is guaranteed to find a globally optimal solution. □

THIS file is intended to serve as a "sample article file" for IEEE journal papers produced under LaTeX using IEEEtran.cls version 1.8b and later. The most common elements are covered in the simplified and updated instructions in "New_IEEEtran_how-to.pdf". For less common elements you can refer back to the original "IEEEtran_HOWTO.pdf". It is assumed that the reader has a basic working knowledge of LaTeX. Those who are new to LaTeX are encouraged to read Tobias Oetiker's "The Not So Short Introduction to LaTeX," available at: http://tug.ctan.org/info/lshort/english/lshort.pdf which provides an overview of working with LaTeX.

## X. EXPERIMENTS

## XI. THE DESIGN, INTENT, AND LIMITATIONS OF THE TEMPLATES

The templates are intended to **approximate the final look and page length of the articles/papers**. **They are NOT intended to be the final produced work that is displayed in print or on IEEEXplore®**. They will help to give the authors an approximation of the number of pages that will be in the final version. The structure of the LaTeX files, as designed, enable easy conversion to XML for the composition systems used by the IEEE. The XML files are used to produce the final print/IEEEXplore pdf and then converted to HTML for IEEEXplore.

## XII. WHERE TO GET LATEX HELP — USER GROUPS

The following online groups are helpful to beginning and experienced LaTeX users. A search through their archives can provide many answers to common questions.

http://www.latex-community.org/
https://tex.stackexchange.com/

## XIII. OTHER RESOURCES

See [**?**], [**?**], [**?**], [**?**], [**?**] for resources on formatting math into text and additional help in working with LaTeX.

## XIV. TEXT

For some of the remainer of this sample we will use dummy text to fill out paragraphs rather than use live text that may violate a copyright.

Itam, que ipiti sum dem velit la sum et dionet quatibus apitet voloritet audam, qui aliciant voloreicid quaspe volorem ut maximusandit faccum conemporerum aut ellatur, nobis arcimus. Fugit odi ut pliquia incitium latum que cusapere perit molupta eaquaeria quod ut optatem poreiur? Quiaerr ovitior suntiant litio bearciur?

Onseque sequaes rectur autate minullore nusae nestiberum, sum voluptatio. Et ratem sequiam quaspername nos rem repudandae volum consequis nos eium aut as molupta tectum ulparumquam ut maximillesti consequas quas inctia cum volectinusa porrum unt eius cusaest exeritatur? Nias es enist fugit pa vollum reium essusam nist et pa aceaqui quo elibusdandis deligendus que nullaci lloreri bla que sa coreriam explacc atiumquos simolorpore, non prehendunt lam que occum [**?**] si aut aut maximus eliaeruntia dia sequiamenime natem sendae ipidemp orehend uciisi omnienetus most verum, ommolendi omnimus, est, veni aut ipsa volendelist mo conserum volores estisciis recessi nveles ut poressitatur sitiis ex endi diti volum dolupta aut aut odi as eatquo cullabo remquis toreptum et des accus dolende pores sequas dolores tinust quas expel moditae ne sum quiatis nis endipie nihilis etum fugiae audi dia quiasit quibus. Ibus el et quatemo luptatque doluptaest et pe volent rem ipidusa eribus utem venimolorae dera qui acea quam etur aceruptat. Gias anis doluptaspic tem et aliquis alique inctiuntiur?

Sedigent, si aligend elibuscid ut et ium volo tem eictore pellore ritatus ut ut ullatus in con con pere nos ab ium di tem aliqui od magnit repta volectur suntio. Nam isquiante doluptis essit, ut eos suntionsecto debitiur sum ea ipitiis adipit, oditiore, a dolorerempos aut harum ius, atquat.

Rum rem ditinti sciendunti volupiciendi sequiae nonsect oreniatur, volores sition ressimil inus solut ea volum harumqui to see(39) mint aut quat eos explis ad quodi debis deliqui aspel earcius.

$$x = \sum_{i=0}^{n} 2iQ. \tag{39}$$

Alis nime volorempera perferi sitio denim repudae pre ducilit atatet volecte ssimillorae dolore, ut pel ipsa nonsequiam in re nus maiost et que dolor sunt eturita tibusanis eatent a aut et dio blaudit reptibu scipitem liquia consequodi od unto ipsae. Et enitia vel et experferum quiat harum sa net faccae dolut voloria nem. Bus ut labo. Ita eum repraer rovitia samendit aut et volupta tecupti busant omni quiae porro que nossimodic temquis anto blacita conse nis am, que ereperum eumquam quaescil imenisci quae magnimos recus ilibeaque cum etum iliate prae parumquatemo blaceaquiam quundia dit apienditem rerit re eici quaes eos sinvers pelecabo. Namendignis as exerupit aut magnim ium illabor roratecte plic tem res apiscipsam et vernat untur a deliquaest que non cus eat ea dolupiducim fugiam volum hil ius dolo eaquis sitis aut landesto quo corerest et auditaquas ditae voloribus, qui optaspis exero cusa am, ut plibus.

## XV. Some Common Elements

### A. Sections and Subsections

Enumeration of section headings is desirable, but not required. When numbered, please be consistent throughout the article, that is, all headings and all levels of section headings in the article should be enumerated. Primary headings are designated with Roman numerals, secondary with capital letters, tertiary with Arabic numbers; and quaternary with lowercase letters. Reference and Acknowledgment headings are unlike all other section headings in text. They are never enumerated. They are simply primary headings without labels, regardless of whether the other headings in the article are enumerated.

### B. Citations to the Bibliography

The coding for the citations is made with the LaTeX `\cite` command. This will display as: see [**?**].

For multiple citations code as follows: `\cite{ref1,ref2,ref3}` which will produce [**?**], [**?**], [**?**]. For reference ranges that are not consecutive code as `\cite{ref1,ref2,ref3,ref9}` which will produce [**?**], [**?**], [**?**], [**?**]

### C. Lists

In this section, we will consider three types of lists: simple unnumbered, numbered, and bulleted. There have been many options added to IEEEtran to enhance the creation of lists. If your lists are more complex than those shown below, please refer to the original "IEEEtran_HOWTO.pdf" for additional options.

**A plain unnumbered list:**

bare_jrnl.tex
bare_conf.tex
bare_jrnl_compsoc.tex
bare_conf_compsoc.tex
bare_jrnl_comsoc.tex

**A simple numbered list:**

1) bare_jrnl.tex
2) bare_conf.tex
3) bare_jrnl_compsoc.tex
4) bare_conf_compsoc.tex
5) bare_jrnl_comsoc.tex

**A simple bulleted list:**

- bare_jrnl.tex
- bare_conf.tex
- bare_jrnl_compsoc.tex
- bare_conf_compsoc.tex
- bare_jrnl_comsoc.tex

### D. Figures

Fig. 1 is an example of a floating figure using the graphicx package. Note that `\label` must occur AFTER (or within) `\caption`. For figures, `\caption` should occur after the `\includegraphics`.

Fig. 2(a) and 2(b) is an example of a double column floating figure using two subfigures. (The subfig.sty package must be loaded for this to work.) The subfigure `\label` commands are set within each subfloat command, and the `\label` for the overall figure must come after `\caption`. `\hfil` is used as a separator to get equal spacing. The combined width of all the parts of the figure should do not exceed the text width or a line break will occur.

Fig. 1. Simulation results for the network.

TABLE I
AN EXAMPLE OF A TABLE

| One | Two |
|---|---|
| Three | Four |

Note that often IEEE papers with multi-part figures do not place the labels within the image itself (using the optional argument to `\subfloat[]`), but instead will reference/describe all of them (a), (b), etc., within the main caption. Be aware that for subfig.sty to generate the (a), (b), etc., subfigure labels, the optional argument to `\subfloat` must be present. If a subcaption is not desired, leave its contents blank, e.g.,`\subfloat[]`.

## XVI. TABLES

Note that, for IEEE-style tables, the `\caption` command should come BEFORE the table. Table captions use title case. Articles (a, an, the), coordinating conjunctions (and, but, for, or, nor), and most short prepositions are lowercase unless they are the first or last word. Table text will default to `\footnotesize` as the IEEE normally uses this smaller font for tables. The `\label` must come after `\caption` as always.

## XVII. ALGORITHMS

Algorithms should be numbered and include a short title. They are set off from the text with rules above and below the title and after the last line.

**Algorithm 1** Weighted Tanimoto ELM.

TRAIN($\mathbf{XT}$)
  **select randomly** $W \subset \mathbf{X}$
  $N_{\mathbf{t}} \leftarrow |\{i : \mathbf{t}_i = \mathbf{t}\}|$ **for** $\mathbf{t} = -1, +1$
  $B_i \leftarrow \sqrt{\text{MAX}(N_{-1}, N_{+1})/N_{\mathbf{t}_i}}$ **for** $i = 1, ..., N$
  $\hat{\mathbf{H}} \leftarrow B \cdot (\mathbf{X}^T\mathbf{W})/(\not\Vdash\mathbf{X} + \not\Vdash\mathbf{W} - \mathbf{X}^T\mathbf{W})$
  $\beta \leftarrow \left(I/C + \hat{\mathbf{H}}^T\hat{\mathbf{H}}\right)^{-1} (\hat{\mathbf{H}}^T B \cdot \mathbf{T})$
  **return** $\mathbf{W}, \beta$

PREDICT($\mathbf{X}$)
  $\mathbf{H} \leftarrow (\mathbf{X}^T\mathbf{W})/(\not\Vdash\mathbf{X} + \not\Vdash\mathbf{W} - \mathbf{X}^T\mathbf{W})$
  **return** SIGN($\mathbf{H}\beta$)

Que sunt eum lam eos si dic to estist, culluptium quid qui nestrum nobis reiumquiatur minimus minctem. Ro moluptat fuga. Itatquiam ut laborpo rersped exceres vollandi repudaerem. Ulparci sunt, qui doluptaquis sumquia ndestiu sapient iorepella sunti veribus. Ro moluptat fuga. Itatquiam ut laborpo rersped exceres vollandi repudaerem.

## XVIII. MATHEMATICAL TYPOGRAPHY AND WHY IT MATTERS

Typographical conventions for mathematical formulas have been developed to **provide uniformity and clarity of presentation across mathematical texts**. This enables the readers of those texts to both understand the author's ideas and to grasp new concepts quickly. While software such as LaTeX and MathType® can produce aesthetically pleasing math when used properly, it is also very easy to misuse the software, potentially resulting in incorrect math display.

IEEE aims to provide authors with the proper guidance on mathematical typesetting style and assist them in writing the best possible article. As such, IEEE has assembled a set of examples of good and bad mathematical typesetting [**?**], [**?**], [**?**], [**?**], [**?**].

Further examples can be found at http://journals.ieeeauthorcenter.ieee.org/wp-content/uploads/sites/7/IEEE-Math-Typesetting-Guide-for-LaTeX-Users.pdf

### A. Display Equations

The simple display equation example shown below uses the "equation" environment. To number the equations, use the `\label` macro to create an identifier for the equation. LaTeX will automatically number the equation for you.

$$x = \sum_{i=0}^{n} 2iQ. \tag{40}$$

is coded as follows:

```
\begin{equation}
\label{deqn_ex1}
x = \sum_{i=0}^{n} 2{i} Q.
\end{equation}
```

To reference this equation in the text use the `\ref` macro. Please see (40)
is coded as follows:

```
Please see (\ref{deqn_ex1})
```

(a) (b)

Fig. 2. Dae. Ad quatur autat ut porepel itemoles dolor autem fuga. Bus quia con nessunti as remo di quatus non perum que nimus. (a) Case I. (b) Case II.

### *B. Equation Numbering*

**Consecutive Numbering:** Equations within an article are numbered consecutively from the beginning of the article to the end, i.e., (1), (2), (3), (4), (5), etc. Do not use roman numerals or section numbers for equation numbering.
**Appendix Equations:** The continuation of consecutively numbered equations is best in the Appendix, but numbering as (A1), (A2), etc., is permissible.

**Hyphens and Periods**: Hyphens and periods should not be used in equation numbers, i.e., use (1a) rather than (1-a) and (2a) rather than (2.a) for subequations. This should be consistent throughout the article.

### *C. Multi-Line Equations and Alignment*

Here we show several examples of multi-line equations and proper alignments.
**A single equation that must break over multiple lines due to length with no specific alignment.**

$$\begin{multline}\text{The first line of this example}\\ \text{The second line of this example}\\ \text{The third line of this example}\end{multline} \tag{41}$$

is coded as:

```
\begin{multline}
\text{The first line of this example}\\
\text{The second line of this example}\\
\text{The third line of this example}
\end{multline}
```

**A single equation with multiple lines aligned at the = signs**

$$a = c + d \tag{42}$$
$$b = e + f \tag{43}$$

is coded as:

```
\begin{align}
a &= c+d \\
b &= e+f
\end{align}
```

The `align` environment can align on multiple points as shown in the following example:

$$x = y \qquad X = Y \qquad a = bc \tag{44}$$
$$x' = y' \qquad X' = Y' \qquad a' = bz \tag{45}$$

is coded as:

```
\begin{align}
x &= y & X & =Y & a &=bc\\
x' &= y' & X' &=Y' &a' &=bz
\end{align}
```

### *D. Subnumbering*

The amsmath package provides a `subequations` environment to facilitate subnumbering. An example:

$$f = g \tag{46a}$$
$$f' = g' \tag{46b}$$
$$\mathcal{L}f = \mathcal{L}g \tag{46c}$$

is coded as:

```
\begin{subequations}\label{eq:2}
\begin{align}
f&=g \label{eq:2A}\\
f' &=g' \label{eq:2B}\\
\mathcal{L}f &= \mathcal{L}g \label{eq:2c}
\end{align}
\end{subequations}
```

### *E. Matrices*

There are several useful matrix environments that can save you some keystrokes. See the example coding below and the output.

**A simple matrix:**

$$\begin{matrix} 0 & 1 \\ 1 & 0 \end{matrix} \tag{47}$$

is coded as:

```
\begin{equation}
\begin{matrix}  0 &  1 \\
1 &  0 \end{matrix}
\end{equation}
```

**A matrix with parenthesis**

$$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{48}$$

is coded as:

```
\begin{equation}
\begin{pmatrix} 0 & -i \\
 i &  0 \end{pmatrix}
\end{equation}
```

**A matrix with square brackets**

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{49}$$

is coded as:

```
\begin{equation}
\begin{bmatrix} 0 & -1 \\
1 &  0 \end{bmatrix}
\end{equation}
```

**A matrix with curly braces**

$$\begin{Bmatrix} 1 & 0 \\ 0 & -1 \end{Bmatrix} \tag{50}$$

is coded as:

```
\begin{equation}
\begin{Bmatrix} 1 &  0 \\
0 & -1 \end{Bmatrix}
\end{equation}
```

**A matrix with single verticals**

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} \tag{51}$$

is coded as:

```
\begin{equation}
\begin{vmatrix} a &  b \\
c &  d \end{vmatrix}
\end{equation}
```

**A matrix with double verticals**

$$\begin{Vmatrix} i & 0 \\ 0 & -i \end{Vmatrix} \tag{52}$$

is coded as:

```
\begin{equation}
\begin{Vmatrix} i &  0 \\
0 & -i \end{Vmatrix}
\end{equation}
```

### *F. Arrays*

The `array` environment allows you some options for matrix-like equations. You will have to manually key the fences, but there are other options for alignment of the columns and for setting horizontal and vertical rules. The argument to `array` controls alignment and placement of vertical rules.

A simple array

$$\left( \begin{array}{cccc} a+b+c & uv & x-y & 27 \\ a+b & u+v & z & 134 \end{array} \right) \tag{53}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{cccc}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

A slight variation on this to better align the numbers in the last column

$$\left( \begin{array}{cccc} a+b+c & uv & x-y & 27 \\ a+b & u+v & z & 134 \end{array} \right) \tag{54}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{cccr}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

An array with vertical and horizontal rules

$$\left( \begin{array}{c|c|c|c} a+b+c & uv & x-y & 27 \\ \hline a+b & u+v & z & 134 \end{array} \right) \tag{55}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{c|c|c|r}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

Note the argument now has the pipe "|" included to indicate the placement of the vertical rules.

### *G. Cases Structures*

Many times cases can be miscoded using the wrong environment, i.e., `array`. Using the `cases` environment will save keystrokes (from not having to type the `\left\lbrace`) and automatically provide the correct column alignment.

$$z_m(t) = \begin{cases} 1, & \text{if } \beta_m(t) \\ 0, & \text{otherwise.} \end{cases}$$

is coded as follows:

```
\begin{equation*}
{z_m(t)} =
\begin{cases}
1,&{\text{if}}\ {\beta }_m(t),\\
{0,}&{\text{otherwise.}}
\end{cases}
\end{equation*}
```

Note that the "&" is used to mark the tabular alignment. This is important to get proper column alignment. Do not use `\quad` or other fixed spaces to try and align the columns. Also, note the use of the `\text` macro for text elements such as "if" and "otherwise."

### H. Function Formatting in Equations

Often, there is an easy way to properly format most common functions. Use of the `\` in front of the function name will in most cases, provide the correct formatting. When this does not work, the following example provides a solution using the `\text` macro:

$$d_R^{KM} = \arg\min_{d_l^{KM}} \{d_1^{KM}, \ldots, d_6^{KM}\}.$$

is coded as follows:

```
\begin{equation*}
 d_{R}^{KM} = \underset {d_{l}^{KM}}
 {\text{arg min}} \{ d_{1}^{KM},
 \ldots,d_{6}^{KM}\}.
\end{equation*}
```

### I. Text Acronyms Inside Equations

This example shows where the acronym "MSE" is coded using `\text{}` to match how it appears in the text.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

```
\begin{equation*}
 \text{MSE} = \frac {1}{n}\sum _{i=1}^{n}
(Y_{i} - \hat {Y_{i}})^{2}
\end{equation*}
```

## XIX. Conclusion

The conclusion goes here.


## Acknowledgments

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.


## Appendix
## Proof of the Zonklar Equations

Use `\appendix` if you have a single appendix: Do not use `\section` anymore after `\appendix`, only `\section*`. If you have multiple appendixes use `\appendices` then use `\section` to start each appendix. You must declare a `\section` before using any `\subsection` or using `\label` (`\appendices` by itself starts a section numbered zero.)

## References Section

You can use a bibliography generated by BibTeX as a .bbl file. BibTeX documentation can be easily obtained at: http://mirror.ctan.org/biblio/bibtex/contrib/doc/ The IEEEtran BibTeX style support page is: http://www.michaelshell.org/tex/ieeetran/bibtex/

## BIOGRAPHY SECTION

If you have an EPS/PDF photo (graphicx package needed), extra braces are needed around the contents of the optional argument to biography to prevent the LaTeX parser from getting confused when it sees the complicated `\includegraphics` command within an optional argument. (You can create your own custom macro containing the `\includegraphics` command to make things simpler here.)

**If you include a photo:**

**Michael Shell** Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

**If you will not include a photo:**

**John Doe** Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.

## REFERENCES


[1] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, "Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints," *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.

[2] B. Li and H. Ma, "Double-deck multi-agent pickup and delivery: Multi-robot rearrangement in large-scale warehouses," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3701–3708, 2023.

[3] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, p. 111377, 2024.

[4] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT Press, 2008.

[5] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Decomposition of finite LTL specifications for efficient multi-agent planning," in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.

[6] L. Li, Z. Zhou, Z. Chen, H. Wang, Z. Kan, and J. Qin, "A formal framework for reactive heterogeneous multirobot task allocation in uncertain semantic environments," *IEEE Transactions on Cybernetics*, pp. 1–14, 2026.

[7] X. Luo and M. M. Zavlanos, "Temporal logic task allocation in heterogeneous multirobot systems," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3602–3621, 2022.

[8] K. Leahy, Z. Serlin, C.-I. Vasile, A. Schoer, A. M. Jones, R. Tron, and C. Belta, "Scalable and robust algorithms for task-based coordination from high-level specifications (scratches)," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2516–2535, 2022.

[9] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.

[10] Y. Kantaros and M. M. Zavlanos, "Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.

[11] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.

[12] P. Gastin and D. Oddoux, "Fast ltl to büchi automata translation," in *International Conference on Computer Aided Verification*, 2001.

[13] Z. Chen, Z. Zhou, S. Wang, J. Li, and Z. Kan, "Fast temporal logic mission planning of multiple robots: A planning decision tree approach," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6146–6153, 2024.

[14] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.

[15] M. Guo and D. V. Dimarogonas, "Multi-agent plan reconfiguration under local LTL specifications," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.

[16] P. Yu and D. V. Dimarogonas, "Distributed motion coordination for multirobot systems under ltl specifications," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 1047–1062, 2022.

[17] S. L. Smith, J. Tůmová, C. Belta, and D. Rus, "Optimal path planning under temporal logic constraints," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.

[18] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.

[19] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Multi-objective search for optimal multi-robot planning with finite ltl specifications and resource constraints," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 768–774.

[20] S. Karaman and E. Frazzoli, "Vehicle routing with linear temporal logic specifications: Applications to multi-uav mission planning," in *AIAA guidance, navigation and control conference and exhibit*, 2008, p. 6838.

[21] X. Luo and C. Liu, "Simultaneous task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Transactions on Robotics*, 2025.

**[22] L. Li, Z. Chen, H. Wang, and Z. Kan, "Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities," *IEEE Transactions on Automation Science and Engineering*, vol. 22, no. 000, pp. 14 030–14 047, 2025.**